DATA501 Final Report

Armand Surbakti, Student ID: **300680133**

Repository Link: https://github.com/ArmandFS/data501_package

1. Introduction and Motivation

Automated detection of machine-generated text is usually framed as a classification problem, which needs a really large labelled dataset of both classes and it generalises poorly to new models. The package that I have developed, "**styloprofile"** instead follows the feature-based framing of [1]. It describes typical human writing with interpretable stylometric features, summarizes a reference corpus into a statistical profile, and then it measures how far a new document sits from it via Mahalanobis distance [2] and an angular distance derived from cosine similarity. The key take-away here is that a document is never classified as machine generated, it is only reported as statistically unusual relative to its reference corpus, a narrower claim that needs no labelled training data. To preface this report, I did use a significant amount of AI, but it was mostly used to guide, give advice for tests, and help code more efficiently.

2. Package Structure

The package structure is as illustrated as follows in the following diagram:

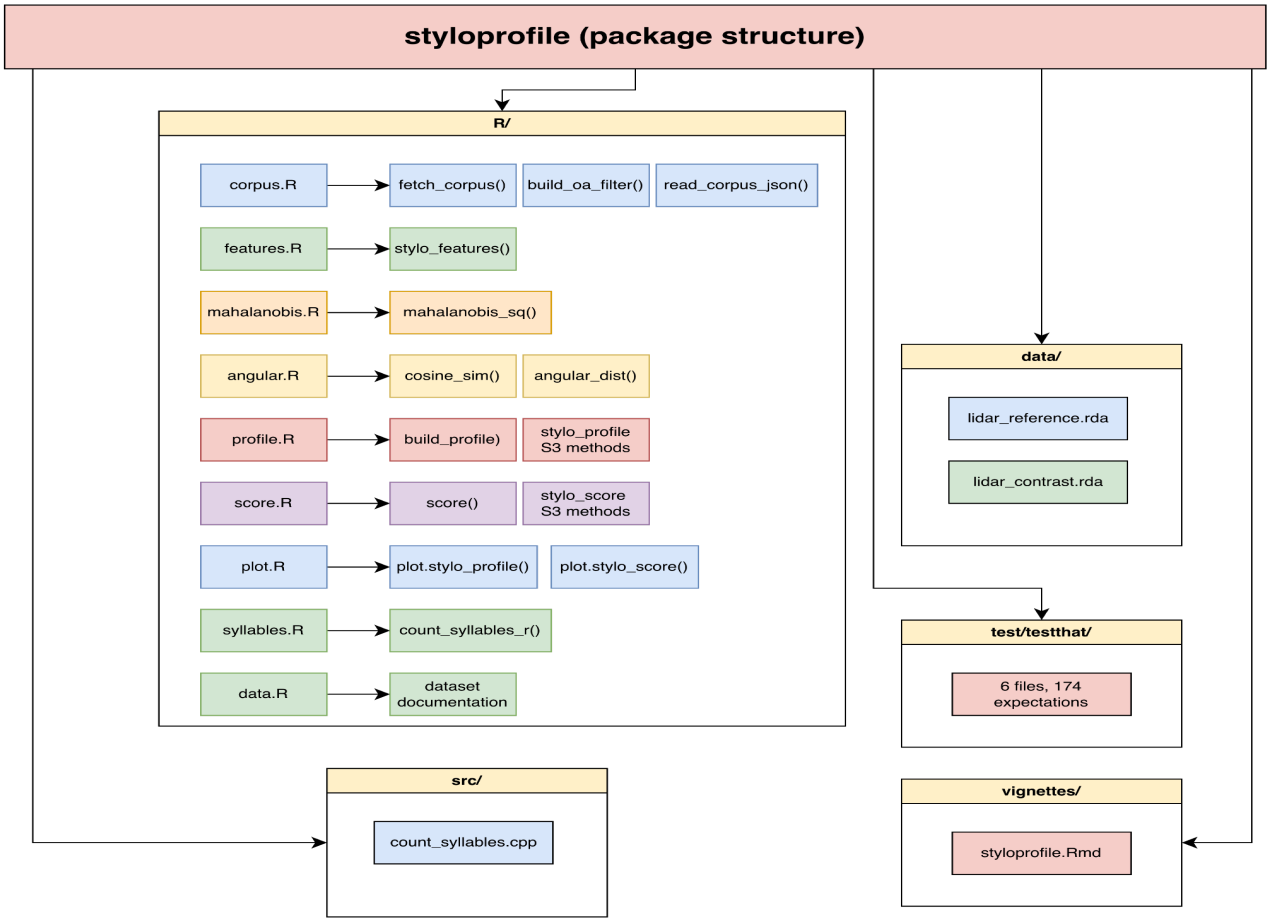


Figure 2.1 styloprofile package structure diagram.

This package diagram/structure here is visualized by what each thing does and its responsibilities. **Corpus.R** handles everything related to acquiring a reference corpus from OpenAlex; **features.R** turns the raw text into the five stylometric features which the rest of the package depends on; **mahalanobis.R** and **angular.R** implements the two distance measures as independent and testable functions, and **profile.R** and **score.R** sits on top of all of them as the object-oriented layer, which produces the **stylo_profile** and **stylo_score** S3 classes which the user actually interacts with. The directory **plot.R** adds visualization methods for both classes, and **syllables.R** holds the pure-R reference implementation that the compiled Rcpp routine in **src/count_syllables.cpp** is tested against. This keeps each file independently testable, and the six files under **tests/testthat/,** which covers 174 tests. The datasets are bundled in **data/** with a walkthrough of the whole package in **vignettes/**.

3. OpenAlex and Corpus Building

OpenAlex is a fully open index of scholarly works, maintained as a free alternative to bibliography databases and released under a CC0 licence (**no paywall, no authentication, no restriction on systematic use) [3]**. This mattered a lot for building a reference corpus: IEEE Xplore, the publisher of the lidar journal this package draws from, prohibits systematic downloading in its terms of service and violating that can terminate an institution's access entirely. OpenAlex indexes the same publications, including abstracts without restriction. Below is a small illustration on using OpenAlex.

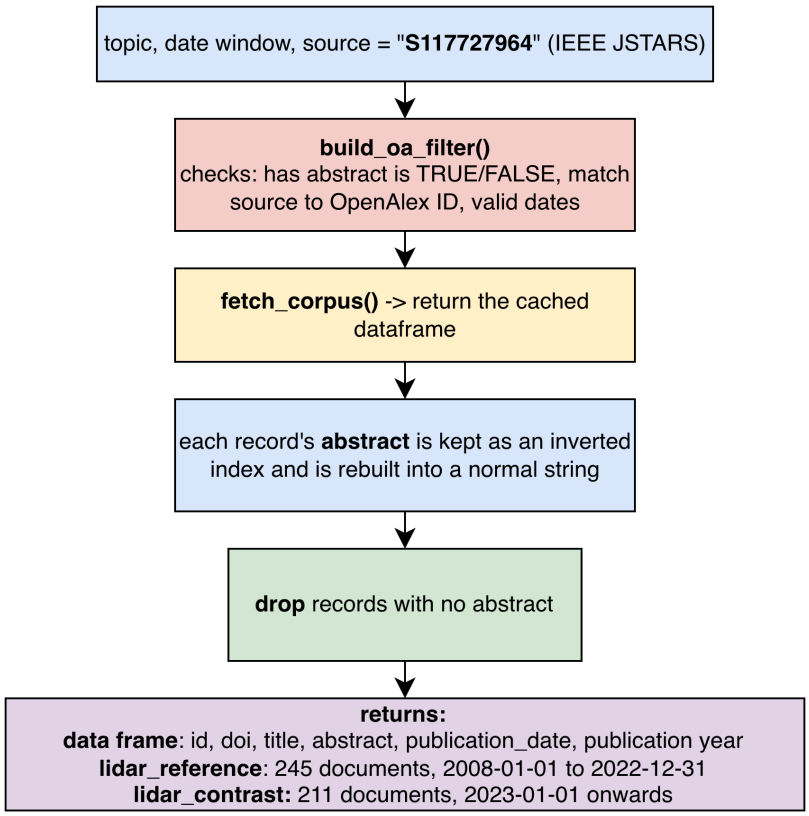


Figure 3.1 OpenAlex retrieval process.

Retrieving the right abstracts from OpenAlex means to scope the corpus. I scoped it into a single journal (with the guide of my supervisor), **the IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing**, requires a specific OpenAlex identifier, which is "**S117727964",** Which is passed as a filter alongside the topic and date window. I had to specify the specific date window and abstract because without it, it returns the corpus that we do not want. The function **build_oa_filter()** builds this filter string, and **fetch_corpus()** sends it to the OpenAlex API, paginating through results and caches them locally, which can be used offline.

4. The Important Methods and Statistical Functions

The main functions used in this package are build_profile**()**, **score()**, **mahalanobis_sq()**, **angular_dist()**. This section summarizes what they do.

4.1 The build_profile() and score() pipeline

These functions split the work into a per document cost. **build_profile()** takes the reference corpus, runs it through **style_features()**, and estimates everything the package will ever need from that corpus, which include the the per-feature mean, per-feature standard deviation, the Cholesky factor of the covariance matrix, and a reference quantile for the angular distance. All of that is bundled into a **stylo_profile** object, computed only once. **score()** then takes any number of new documents, extracts the same five features, and reuses that profile to compute a distance for each one (either using **mahalanobis_sq() or angular_dist()**, and returns a **stylo_score** object. Both objects carry print, summary, and plot methods, and summary(). Below is a diagram of this pipeline.

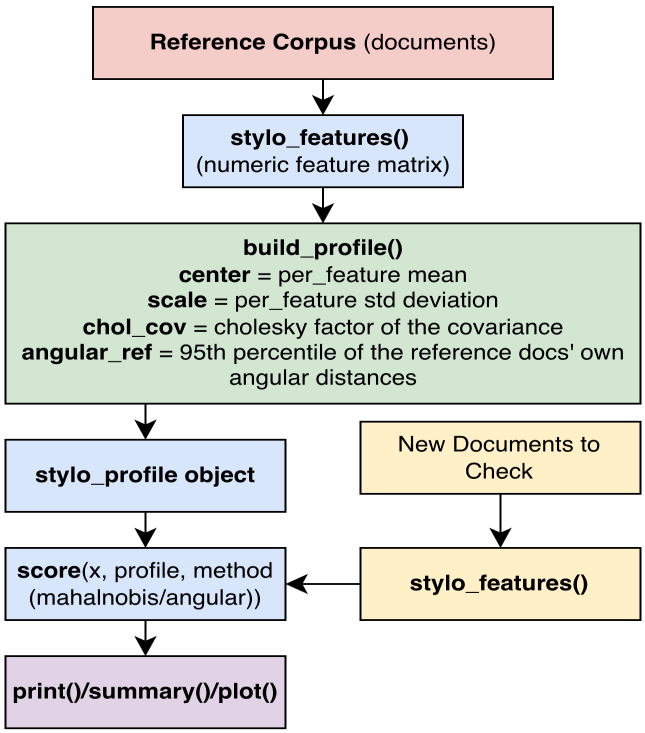


Figure 4.1 Profile building and scoring pipeline

4.2 Mahalanobis Distance Function

The function **mahalanobis_sq()** computes the squared Mahalanobis distance without inverting the covariance matrix. It takes the covariance's Cholesky factor instead, solves one triangular linear system per document, and sums the squared result. Some benchmarking showed that the explicit-inverse formulation is marginally faster at the five features this package uses, and the cholesky route only overtakes it from around 25 features upward. This is kept anyway for two reasons that don't depend on speed. The factorization happens once, in **build_profile()** and is reused for every document **score()** processes afterward. The **chol()** function fails on a matrix that isn't positive definite, which turns a meaningless result into an error, which matters given that the covariance from this five-feature, 245 document corpus is close to singular (condition number 179,871.6 unshrunk, 45.9 with shrink = 0.1). The diagram below shows this process.

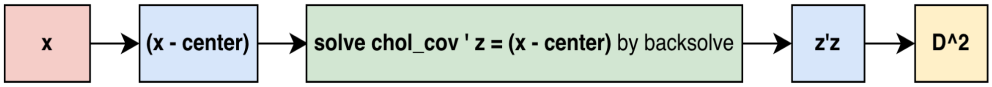


Figure 4.2 Mahalanobis Distance Diagram.

4.3 Angular Distance Function

The function **angular_dist()** answers a different question than Mahalanobis distance. It's not about how extreme a document is, but whether the proportions between its features point in the wrong direction, even when none of them is individually unusual. Each feature is first divided by its reference standard deviation, so the comparison is of direction rather than magnitude, and the cosine similarity between the scaled document and the scaled reference centre is converted into an angle. I divided the angle by pi, rather than using one minus the cosine similarity because **acos(cos(theta))pi** is a true metric that satisfies the triangle inequality, while one minus the cosine similarity is not. A clamp step guards against floating-point error pushing an exact self-similarity fractionally above 1, which would otherwise make **acos()** return **NaN.** The diagram below shows this process.

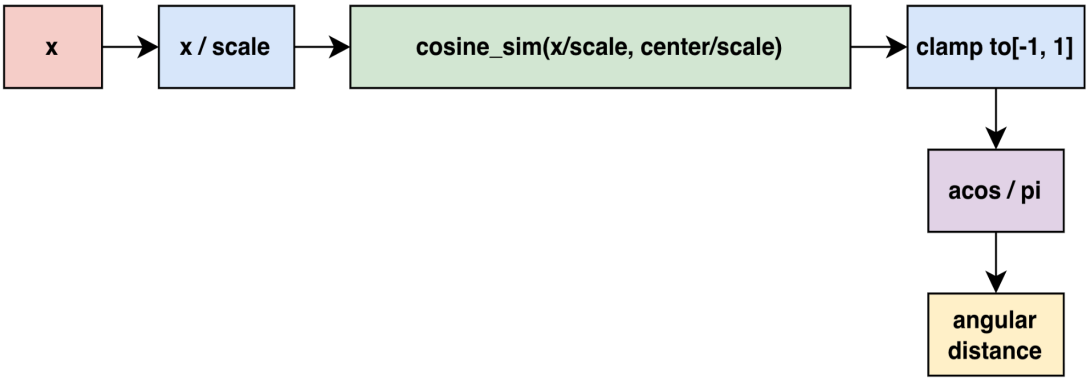


Figure 4.3 Angular Distance Diagram.

5. Testing, Peer Review Fixing, Results, and Functions in Action.

5.1 Automated Testing Results

This section here is about testing and responding to the feedback that the peer review assessment gave me. The current testing suite has 174 passing expectations across 98 **test_that()** blocks in six files, which covers the major components of the package. There are tests that test feature extraction, both distance functions, corpus pipeline, the profile/score object oriented-workflow, and the compiled syllable counter of the **Rcpp** against its R-equivalent. **R CMD check** passes at zero errors and zero warnings. Below is a table of the testing.

File	Blocks	Expect	Covers
test-angular.R	12	21	cosine_sim, angular_dist
test-corpus.R	27	47	corpus query/parsing
test-features.R	14	20	stylo_features
test-mahalanobis.R	13	22	mahalanobis_sq
test-profile.R	19	38	build_profile, score, S3 methods
test-syllables.R	13	26	count_syllables_cpp/_r
Total	**98**	**174**	

Table 5.1 Automated Testing results.

5.2 Peer Review Fixing

One of the first findings from the peer review assessment that needed to be addressed is that **has_abstract** was supposed to be restricted to **TRUE/FALSE**, but the filter was built with **if(isTRUE(has_abstract))"has_abstract:true"**. Passing **has_abstract = "yes"** is not TRUE, so instead of erroring, the **has_abstract:true** clause disappeared from the query, which means the search could silently start returning papers with no abstract at all. I fixed this by having **build_oa_filter()** validate **has_abstract** as a single, non-NA logical before building anything and stops with **has_bastract must be TRUE or FALSE** otherwise. In the case of **has_abstract = FALSE,** it still correctly omits the clause.

The second finding that the peer review assessment found was that the **source** argument should be a valid OpenAlex source ID such as "**S117727964"**, but nothing checked its shape. Passing **source = 123** was accepted by **build_oa_filter(),** built into a filter string, and only

fetch_corpus() sending that filter to the OpenAlex API exposed the problem as **a bare 400 Bad Request network error**, with no indication from the package itself that source was the cause. To fix this, the **source** shape, and also **publisher** are now validated against their OpenAlex ID pattern before the filter is built, so the request is never sent in the first place.

The final fix was a date window that was entirely in the future being accepted, with no signal. The function **build_oa_filter()** already validates that **from** precedes **to** and that both are real YYYY-MM-DD dates, but never checked either against today's date. A window like from **"2030-01-01"** was accepted, sent to OpenAlex, and came back as a silently empty corpus with no indication that the date range itself, not the topic or journal, was why there were no results. I managed to fix this and build_oa_filter() now gives a warning when **from** is later than today.

5.3 Results

Holding out 80 out of the 254 reference abstracts, It built a profile on the remaining 165, and scored both the held-out pre 2023 documents and all 211 post 2023 documents. Below is a table of the results.

Method	Median, Pre	Median, Post	Flagged, Pre	Flagged, Post	Wilcoxon
Mahalanobis	4.130	6,061	7.5%	17.1%	p = 0.00024
Angular	0.0227	0.0295	6.2%	18.0%	p = 0.000015

Table 5.2 Method Results.

Both methods agree that post-2023 abstracts sit further from the pre-2023 profile, and roughly 2.5× as many are flagged. The features moving most are vocab_sophistication (+0.055) and prop_polysyllabic (+0.037) are rarer, longer words, while sentences are slightly shorter. The contrast corpus is not labelled. It is defined by publication date, not by authorship, so this is a shift in the distribution between two time periods and not a measure of detection accuracy. Cut-offs come from the reference corpus, never from the scores being judged: the chi-squared 95% point on p degrees of freedom for Mahalanobis, and the 95th percentile of the reference documents' own angles for the angular distance.

5.4 Functions in Action

Most of the functions can be tested to the **PACKAGE_DEMO.md** file in the github repository, which walks through every exported function against the bundled **lidarlidar_reference/lidar_contrast corpus:** feature extraction, both distance functions individually, the full **build_profile()** → **score()** workflow with **print()/summary()** on both resulting objects, the two **plot()** methods, and a dedicated section re-running the three exact scenarios the peer review found broken, to demonstrate they are now fixed. Screenshots of a selection of this output, including both plots, are included in the appendix.

6. Reflections, Future Plans, and Limitations

This was a genuinely challenging process for me. I had never done software development in RStudio before this project, so testing, writing a vignette, writing objects, and producing proper **roxygen** documentation were all new skills, which was a steep but rewarding learning curve.

If I were to do this differently, I would decide on the reference corpus much earlier. The original proposal had the corpus be the HC3 and the Brown Corpus, but then that was a much bigger dataset and too general, so I had to pivot to the Lidar only journals only. This switch happened in the middle of the trimester and if it were to happen earlier, I would have time gone into refining the features themselves. The result is that the post-2023 abstracts sit further from the reference profile as much as 2.5x as many flagged and its a distributional shift finding. Deciding on the corpus sooner would have me design the features around the abstract-only corpus from the start, rather than adapting to it midway.

One limitation stayed visible throughout and is worth staying: two of the five features, such as vocabulary and sentence length, correlate with English proficiency and education. Any result from this package is relative to whatever reference corpus it was built against and needs to be documented.

Future plans for this package include incorporating other topics for the corpus. The corpus-building pipeline that I developed only needs an OpenAlex source ID and a topic string, so the next step is expanding past one IEEE journal to many others, across topics that have nothing to do with lidar. Beyond corpus scale, there are some edge cases this project didn't have time to test, such as non-English or mixed-Language text, or documents that are not from an academic context, such as new articles, student essays, forum posts, etc. There is also room for other distance-based methods Mahalanobis and angular distance such as using a distributional distance such as KL divergence that compares to corpora's feature distribution directly, rather than scoring one document at a time against a single reference profile.

7. References

[1] Fröhling, L., & Zubiaga, A. (2021). ***Feature-based detection of automated language models: Tackling GPT-2***, ***GPT-3 and Grover.*** PeerJ Computer Science, 7, e443.

[2] Mahalanobis, P. C. (1936). ***On the generalised distance in statistics.*** Proceedings of the National Institute of Sciences of India, 2(1), 49–55.

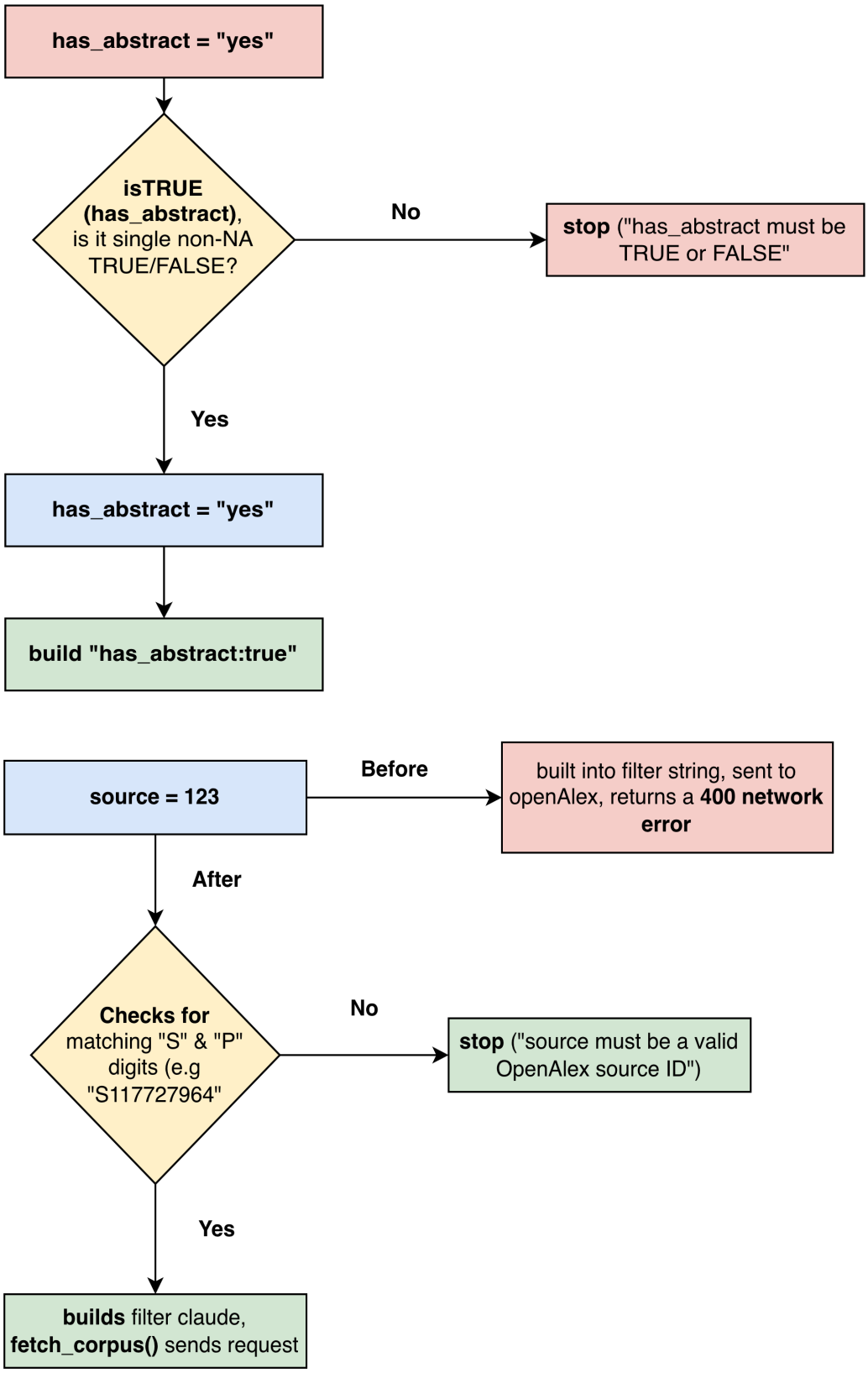
[3] Priem, J., Piwowar, H., & Orr, R. (2022). ***OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts***. arXiv:2205.01833.

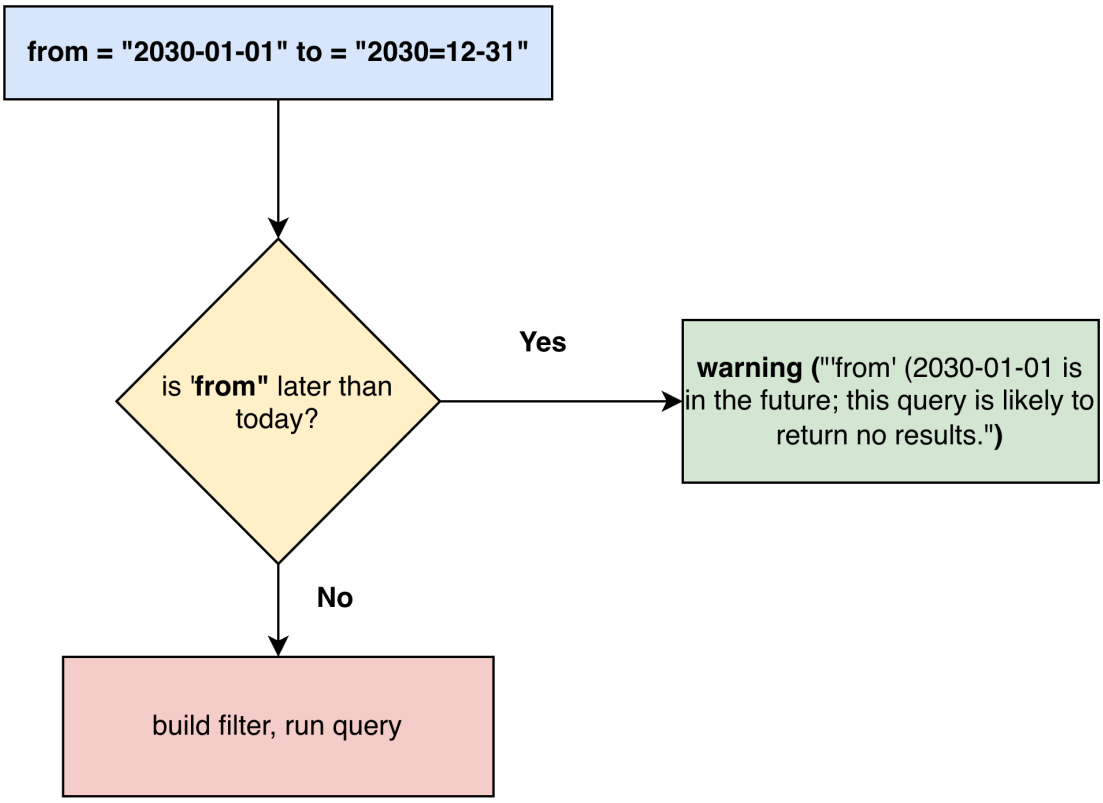
[4] Salton, G., Wong, A., & Yang, C. S. (1975). ***A vector space model for automatic indexing.*** Communications of the ACM, 18(11), 613–620.

[5] van Dongen, S., & Enright, A. J. (2012). ***Metric distances derived from cosine similarity and Pearson and Spearman correlations***. arXiv:1208.3145.

8. Appendix

Fixing Peer Review Assessment Diagrams and Code Testing






```
> build_oa_filter("test", "2030-01-01", "2030-12-31")
[1] "title_and_abstract.search:test,primary_location.source.id:S117727964,type:article,has
_abstract:true,from_publication_date:2030-01-01,to_publication_date:2030-12-31"

Warning message:
`from` (2030-01-01) is in the future; this query is likely to return no results.
```

```
> build_oa_filter("lidar", "2008-01-01", "2022-12-31", has_abstract = "yes")

Error: `has_abstract` must be TRUE or FALSE.
```

```
> build_oa_filter("machine learning", "2020-01-01", "2020-12-31", source = 123)

Error: `source` must be a valid OpenAlex source ID, e.g. "S117727964".
```

Functions in Action

```
> stylo_features(c("This is a simple test sentence.", ""))
     mean_sentence_length conj_rate cv_word_length prop_polysyllabic
[1,]                    6         0      0.6150122                  0
[2,]                   NA        NA             NA                 NA
     vocab_sophistication
[1,]                  0.5
[2,]                   NA
```

```
> angular_dist(c(1, 0), c(1, 1))
[1] 0.25
>
```

```
> profile <- build_profile(lidar_reference$abstract)

 Profile built from 245 documents on 5 features.

> profile
<stylo_profile>
  245 documents, 5 features

       mean_sentence_length conj_rate cv_word_length prop_polysyllabic
center              25.6192    1.7895         0.5432            0.2675
scale               11.1352    0.7722         0.0296            0.0438
       vocab_sophistication
center               0.5763
scale                0.0410
```

```
> summary(profile)
Reference profile

Documents: 245
Features:  5

                          mean      sd
mean_sentence_length 25.6192 11.1352
conj_rate             1.7895  0.7722
cv_word_length        0.5432  0.0296
prop_polysyllabic     0.2675  0.0438
vocab_sophistication  0.5763  0.0410

Covariance condition number: 179871.6
The features are close to collinear; consider build_profile(shrink=).
```

```
> s_maha <- score(lidar_contrast$abstract, profile, "mahalanobis")
> summary(s_maha)
Scores for 211 document(s), mahalanobis distance
Reference profile: 245 documents, 5 features

     0%     25%     50%     75%    100%
 0.4965  3.0095  5.5194  7.8416 36.6638

Mean: 6.3495

Flagged above 11.0705 (chi-squared(5) 95%): 28 of 211 (13.3%)

A flagged document is unusual relative to the reference corpus.
It is not evidence that the document was machine-generated.
```

```
> s_ang <- score(lidar_contrast$abstract, profile, "angular")
> summary(s_ang)
Scores for 211 document(s), angular distance
Reference profile: 245 documents, 5 features

    0%    25%    50%    75%   100%
0.0062 0.0182 0.0273 0.0369 0.0771

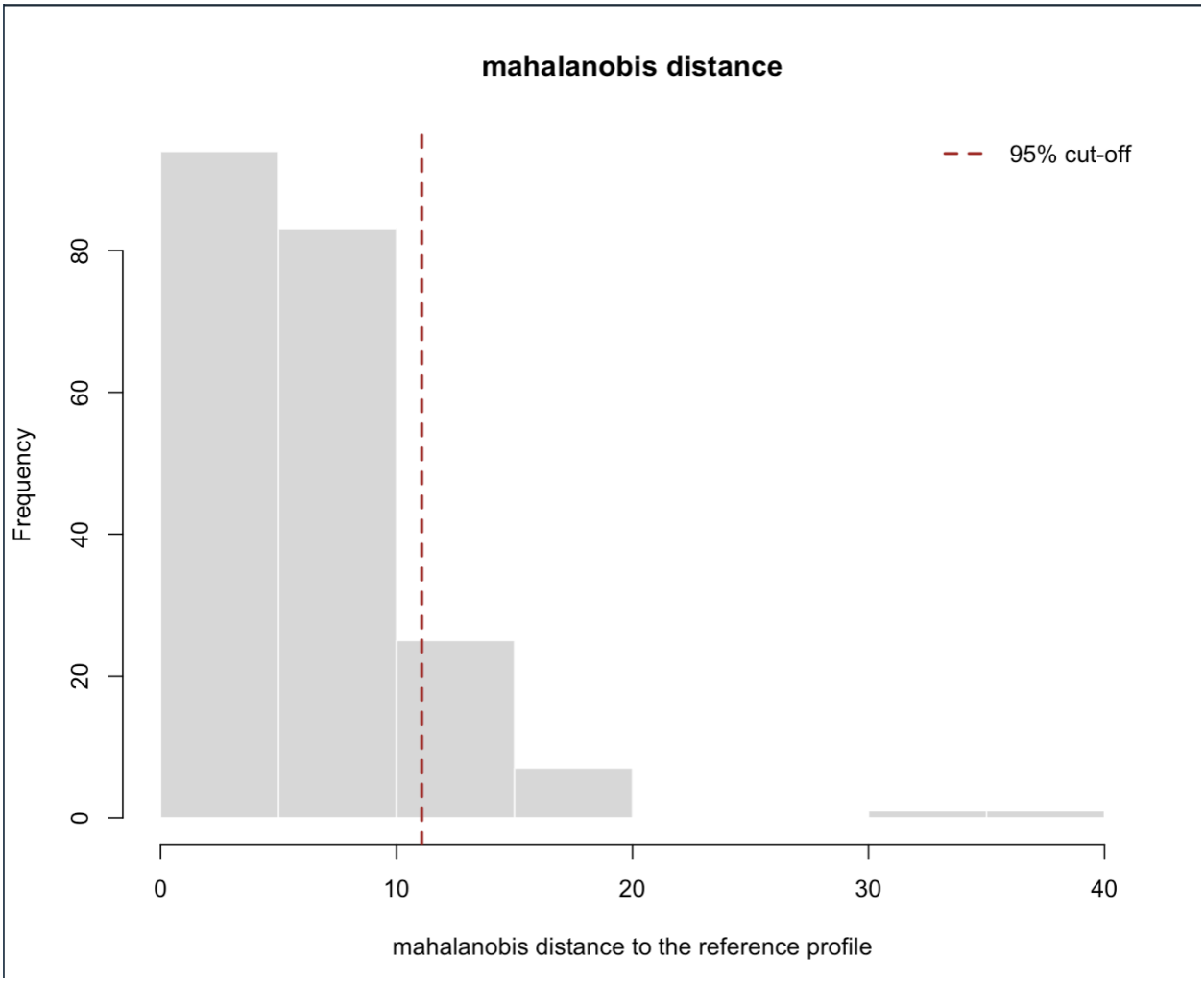
Mean: 0.0287

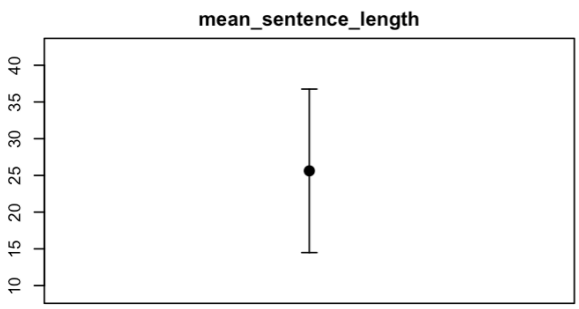
Flagged above 0.0389 (95th percentile of the reference corpus): 42 of 211 (19.9%)

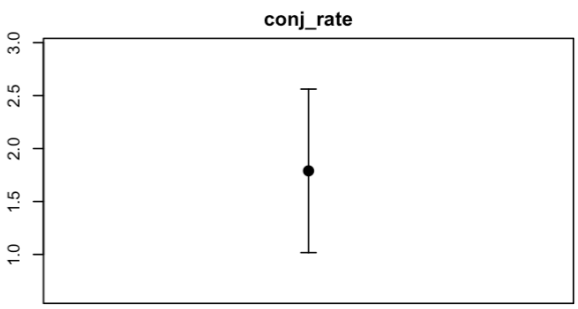
A flagged document is unusual relative to the reference corpus.
It is not evidence that the document was machine-generated.
```

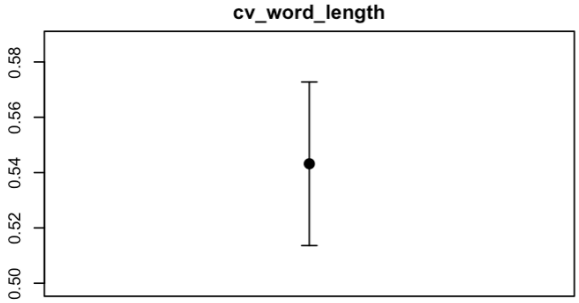
```
> f <- system.file("extdata", "openalex_sample.json", package = "styloprofile")
> read_corpus_json(f)
Parsed 3 records; dropped 2 (no abstract, or under 40 words).
                              id                                     doi
1 https://openalex.org/W4210423514 https://doi.org/10.1109/tro.2022.3141876
                                      title
1 FAST-LIO2: Fast Direct LiDAR-Inertial Odometry

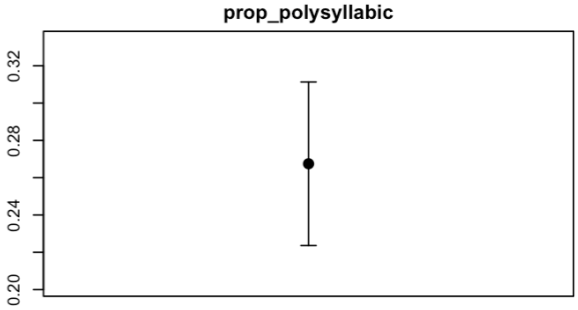
abstract
1 This article presents FAST-LIO2: a fast, robust, and versatile LiDAR-inertial odometry frame
wo key novelties that allow fast, robust, and accurate LiDAR navigation (and mapping). The fir
mapping) without extracting features. This enables the exploitation of subtle features in the
extraction module also makes it naturally adaptable to emerging LiDARs of different scanning p
tree data structure, incremental k-d tree (ikd-Tree), that enables incremental updates (i.e.,
uctures (octree, R$^\\ast$-tree, andnanoflannk-d tree),ikd-Treeachieves superior overall perfo
comparison in 19 sequences from a variety of open LiDAR datasets. FAST-LIO2 achieves consisten
ial navigation systems. Various real-world experiments on solid-state LiDARs with small field
Hz odometry and mapping in large outdoor environments), robust (e.g., reliable pose estimation
le to both multiline spinning and solid-state LiDARs, unmanned aerial vehicle (UAV) and handhe
than existing methods. Our implementation of the system FAST-LIO2 and the data structureikd-Tr
  publication_date publication_year
1       2022-01-31             2022
```

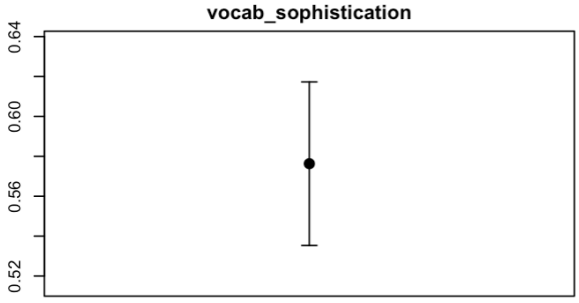



mean_sentence_length
10
15
20
25
30
35
40


conj_rate
1.0
1.5
2.0
2.5
3.0


cv_word_length
0.50
0.52
0.54
0.56
0.58


prop_polysyllabic
0.20
0.24
0.28
0.32


vocab_sophistication
0.52
0.56
0.60
0.64

Getting started with styloprofile

What this package does

`styloprofile` measures how far a document sits from a statistical profile of human writing, rather than training a human-vs-AI classifier. It describes typical writing with five interpretable stylometric features (sentence length, conjunction use, variation in word length, syllable complexity, vocabulary sophistication), summarises a reference corpus into a `stylo_profile`, and scores new documents against it with two complementary distances. A high score means a document is unusual *relative to the reference corpus* – not evidence that it was machine-generated.

This vignette runs end to end using the two corpora bundled with the package, so it needs no network access. For fetching a corpus of your own from OpenAlex, see `?fetch_corpus`.

```
library(styloprofile)
```

1. The bundled corpora

`lidar_reference` and `lidar_contrast` are abstracts of lidar articles from one journal (IEEE JSTARS), retrieved from OpenAlex and split by publication date: `lidar_reference` runs through the end of 2022, `lidar_contrast` from 2023 onward.

```
nrow(lidar_reference)
#> [1] 245
nrow(lidar_contrast)
#> [1] 211
lidar_reference$abstract[1]
#> [1] "The 2013 Data Fusion Contest organized by the Data Fusion Technical Committee (DFTC) of th
```